

## COMP1786 Logbook Upload Template

### Logbook 1

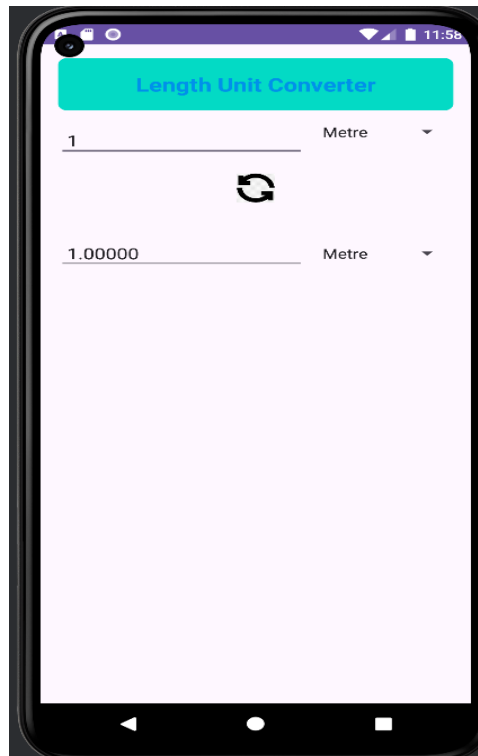
#### 1. Basic Information

|                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1 Student name                                                                                                                                                                      | Vo Tan Nghia                                                                                                                                                                                                                                                                                                                                |
| 1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work. | Do it alone                                                                                                                                                                                                                                                                                                                                 |
| 1.3 Which Exercise is this? Tick as appropriate.                                                                                                                                      | <ul style="list-style-type: none"><li>• Exercise 1 <input checked="" type="checkbox"/></li><li>• Exercise 2 <input type="checkbox"/></li><li>• Exercise 3 <input type="checkbox"/></li></ul>                                                                                                                                                |
| 1.4 How well did you complete the exercise? Tick as appropriate.                                                                                                                      | <ul style="list-style-type: none"><li>• I tried but couldn't complete it <input type="checkbox"/></li><li>• I did it but I feel I should have done better <input type="checkbox"/></li><li>• I did everything that was asked <input type="checkbox"/></li><li>• I did more than was asked for <input checked="" type="checkbox"/></li></ul> |
| 1.5 Briefly explain your answer to question 1.4<br><br>Without any explanation/justification, your scores will be deducted.                                                           | During the implementation, I finished ahead of schedule, met the functional requirements, and added auto-update and migration functionality, but I didn't develop it further because I didn't have much time for it                                                                                                                         |

## 2. Exercise answer

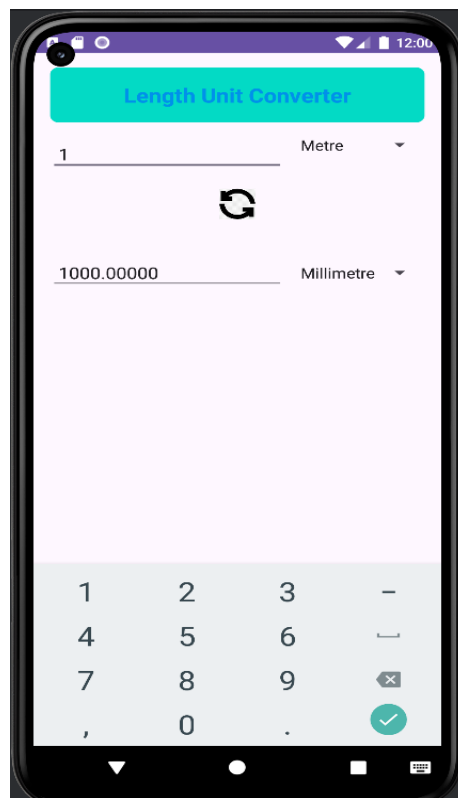
### 2.1 Screen shots demonstrating what you achieved

Figure 1 shows a simple example of converting user input from one length unit to another



*Figure 1: Application that changes length*

Figure 2 is taking an input value of 1 and a meter type. It converts it to an output type of millimeters. The result returned in millimeters is 1000.



*Figure 2: Converting from meters to millimeters*

Tapping on the length unit will show other length units that the user can use

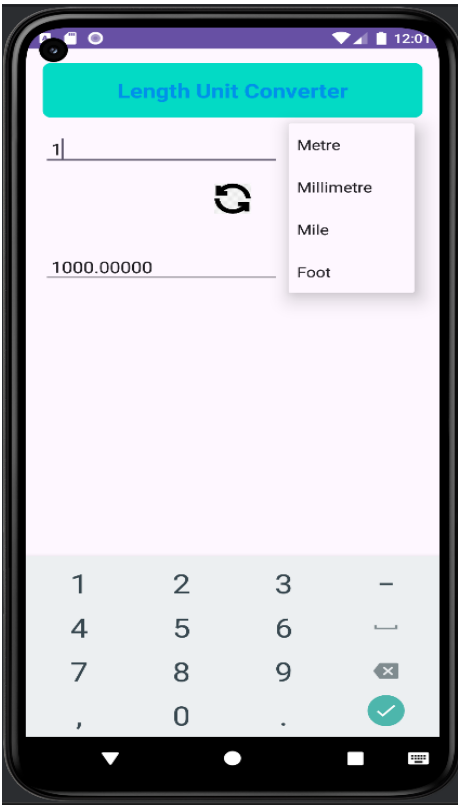


Figure 3: Select length unit types

When we click the switch, it will switch the length units between the input and output. At the same time, it will switch the input and output values to the corresponding length units.

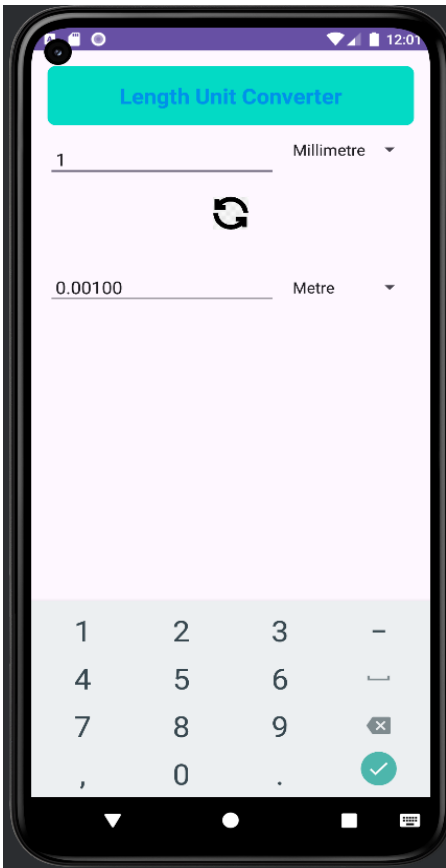


Figure 4: Switch function

## 2.2 Code that you wrote

MainActivity class: MainActivity is the main class in the application, which implements the function of converting unit lengths. The user enters a value in the EditText box, selects the input and output units via two Spinners. The application automatically converts and displays the result in the TextView. When the user changes the input value or unit, the application updates the result immediately. The ImageView button allows the user to reverse the input and output positions.

```
package com.example.lengthunitconverterapplication;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.Spinner;
import android.widget.TextView;
```

```
public class MainActivity extends AppCompatActivity {
    private EditText inputValue;
    private Spinner inputUnit, outputUnit;
    private TextView resultText;
    private ImageView switchUnits;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        inputValue = findViewById(R.id.inputValue);
        inputUnit = findViewById(R.id.inputUnit);
        outputUnit = findViewById(R.id.outputUnit);
        resultText = findViewById(R.id.resultText);
        switchUnits = findViewById(R.id.switchUnits);
```

```
        ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(this,
            R.array.length_units, android.R.layout.simple_spinner_item);
```

```
        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        inputUnit.setAdapter(adapter);
        outputUnit.setAdapter(adapter);
        inputUnit.setSelection(adapter.getPosition("Metre"));
        outputUnit.setSelection(adapter.getPosition("Metre"));
        inputValue.setText("1");
```

```

convertLength();
inputValue.addTextChangedListener(new TextWatcher() {
    @Override
    public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
    @Override
    public void onTextChanged(CharSequence s, int start, int before, int count) {
        convertLength();
    }
    @Override
    public void afterTextChanged(Editable s) {}
});
inputUnit.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parentView, View selectedItemView, int
position, long id) {
        convertLength();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parentView) {}
});
outputUnit.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parentView, View selectedItemView, int
position, long id) {
        convertLength();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parentView) {}
});

switchUnits.setOnClickListener(v -> {
    int inputPosition = inputUnit.getSelectedItemPosition();
    int outputPosition = outputUnit.getSelectedItemPosition();
    inputUnit.setSelection(outputPosition);
    outputUnit.setSelection(inputPosition);
    convertLength();
});
}
private void convertLength() {
    String valueStr = inputValue.getText().toString().trim();
    if (valueStr.isEmpty()) {
        resultText.setText("0");
        return;
    }

    try {
        double value = Double.parseDouble(valueStr);
        String inputUnitStr = inputUnit.getSelectedItem().toString();

```

```

        String outputUnitStr = outputUnit.getSelectedItem().toString();
        double result = convert(value, inputUnitStr, outputUnitStr);
        resultText.setText(String.format("%.5f", result));
    } catch (NumberFormatException e) {
        resultText.setText("0");
    }
}

private double convert(double value, String inputUnit, String outputUnit) {
    double meters = 0.0;
    switch (inputUnit) {
        case "Metre":
            meters = value;
            break;
        case "Millimetre":
            meters = value / 1000;
            break;
        case "Mile":
            meters = value * 1609.34;
            break;
        case "Foot":
            meters = value * 0.3048;
            break;
    }
    switch (outputUnit) {
        case "Metre":
            return meters;
        case "Millimetre":
            return meters * 1000;
        case "Mile":
            return meters / 1609.34;
        case "Foot":
            return meters / 0.3048;
    }
    return 0.0;
}
}

```

activity\_main.xml: This interface uses a LinearLayout in activity\_main.xml to arrange the components vertically. It consists of a title ("Length Unit Converter"), an input field (EditText) for the user to enter a value, two Spinners to select the input and output units, and an ImageView to toggle between units. The conversion result will be displayed in an EditText field that does not allow input.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"

```

```
android:padding="16dp">
```

```
<TextView  
    android:id="@+id/textView"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Length Unit Converter"  
    android:gravity="center"  
    style="@style/TextViewHeading"/>
```

```
<LinearLayout  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="horizontal"  
    android:gravity="center_vertical">
```

```
    <EditText  
        android:id="@+id/inputValue"  
        android:layout_width="0dp"  
        android:layout_height="wrap_content"  
        android:layout_weight="1"  
        android:inputType="numberDecimal"  
        android:padding="8dp"  
        android:layout_marginTop="10dp"  
        android:hint="Enter value" />
```

```
    <Spinner  
        android:id="@+id/inputUnit"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:padding="8dp" />
```

```
</LinearLayout>
```

```
<ImageView  
    android:id="@+id/switchUnits"  
    android:layout_width="50dp"  
    android:layout_height="50dp"  
    android:layout_gravity="center"  
    android:src="@drawable/ic_switch_units"  
    android:contentDescription="Switch Units"  
    android:padding="8dp"  
    android:layout_margin="16dp" />
```

```
<LinearLayout  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="horizontal"  
    android:gravity="center_vertical"
```

```
android:paddingTop="16dp">
```

```
<EditText
    android:id="@+id/resultText"
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:padding="8dp"
    android:hint="Result"
    android:focusable="false" />
```

```
<Spinner
    android:id="@+id/outputUnit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:padding="8dp" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```

colors.xml: This colors.xml file defines the colors used in Android applications. Each color is given a name and assigned a hex color code value.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="red">#D50000</color>
    <color name="colorPrimary">#0091EA</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="colorLable">#373D3A</color>
</resources>
```

strings.xml: This file contains a string resource named app\_name, whose value is "Length unit converter application". This string is used to display the application name and can be referred to as @string/app\_name in interface files or source code.

```
<resources>
    <string name="app_name">Length unit converter application</string>
</resources>
```

themes.xml: This file defines the interface resources for the app, including a main theme that supports dark/light modes, a string array containing measurement units like "Meters" and "Foods," and custom fonts for TextView labels and titles. This makes it easy for the app to customize the look and feel and manage resources.

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.LengthUnitConverterApplication"
        parent="Theme.Material3.DayNight.NoActionBar">
```



```

        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>
    <string-array name="length_units">
        <item>Metre</item>
        <item>Millimetre</item>
        <item>Mile</item>
        <item>Foot</item>
    </string-array>
    <style name="TextViewLabel">
        <item name="android:textColor">@color/colorLable</item> <!-- Change to your desired
color -->
        <item name="android:textSize">20sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:paddingHorizontal">16dp</item> <!-- Add padding for better
spacing -->
        <item name="android:paddingTop">10dp</item>
        <item name="android:paddingBottom">5dp</item>

    </style>

    <style name="TextViewHeading">
        <item name="android:textColor">@color/colorPrimary</item> <!-- Change to your
desired color -->
        <item name="android:textSize">22sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:padding">16dp</item> <!-- Add padding for better spacing -->
        <item name="android:background">@drawable/textview_background</item> <!--
Optional: custom background -->
    </style>

    <style name="Theme.LengthUnitConverterApplication"
parent="Base.Theme.LengthUnitConverterApplication" />
</resources>

```

## Logbook 2

### 1. Basic Information

|                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1 Student name                                                                                                                                                                      | Vo Tan Nghia                                                                                                                                                                                                                                                                                                                                |
| 1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work. | Do it alone                                                                                                                                                                                                                                                                                                                                 |
| 1.3 Which Exercise is this? Tick as appropriate.                                                                                                                                      | <ul style="list-style-type: none"><li>• Exercise 1 <input type="checkbox"/></li><li>• Exercise 2 <input checked="" type="checkbox"/></li><li>• Exercise 3 <input type="checkbox"/></li></ul>                                                                                                                                                |
| 1.4 How well did you complete the exercise? Tick as appropriate.                                                                                                                      | <ul style="list-style-type: none"><li>• I tried but couldn't complete it <input type="checkbox"/></li><li>• I did it but I feel I should have done better <input type="checkbox"/></li><li>• I did everything that was asked <input type="checkbox"/></li><li>• I did more than was asked for <input checked="" type="checkbox"/></li></ul> |
| 1.5 Briefly explain your answer to question 1.4<br>Without any explanation/justification, your scores will be deducted.                                                               | In logbook 2, I finished on schedule with the required functionality such as adding, editing, and deleting tasks in the to-do list. In addition, I added a dialog for users to select a date                                                                                                                                                |

### 2. Exercise answer

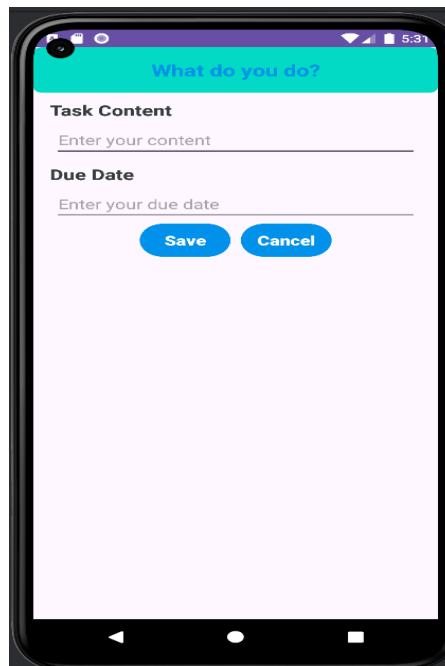
#### 2.1 Screen shots demonstrating what you achieved

Figure 5 is the show task list interface.



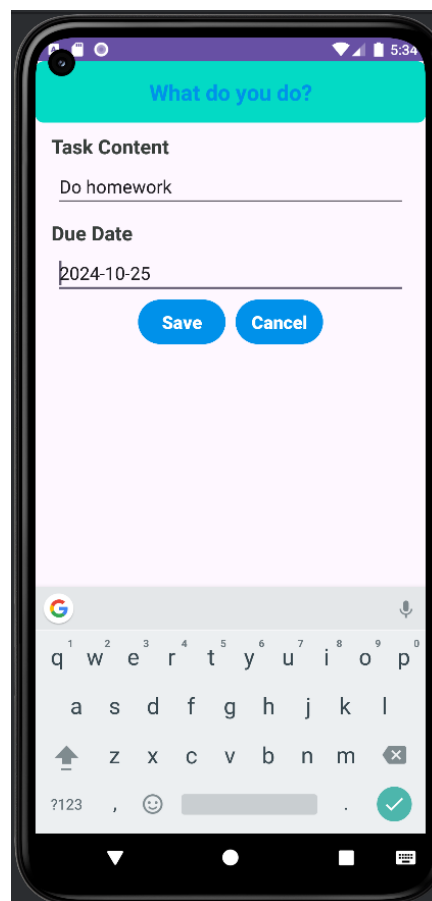
Figure 5: Logbook 2 task list interface

Figure 6 is the interface of the task addition page when the user clicks the Add button.



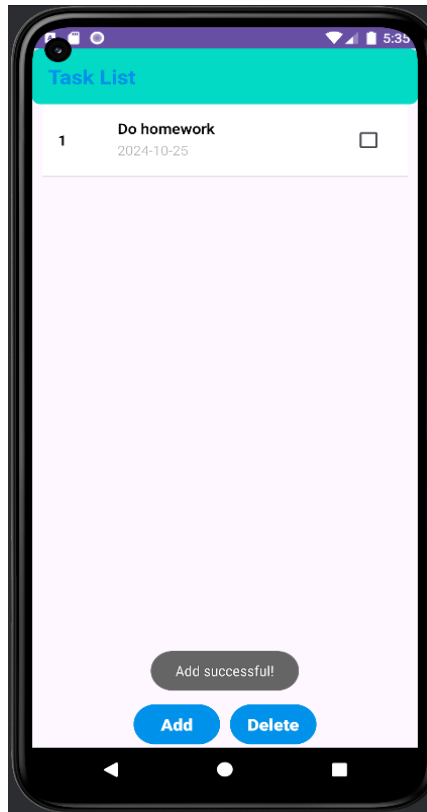
*Figure 6: Add logbook 2 task interface*

After the user goes to the add task page, fill in all the required information as shown in image 7 below.



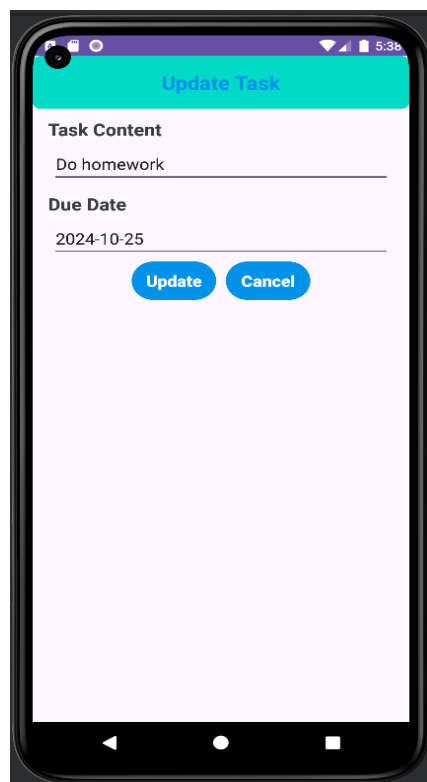
*Figure 7: Fill in data in logbook 2 task add page*

After filling in all the information in figure 7, click the save button, the task will be saved and a successful add notification will appear as in figure 8.



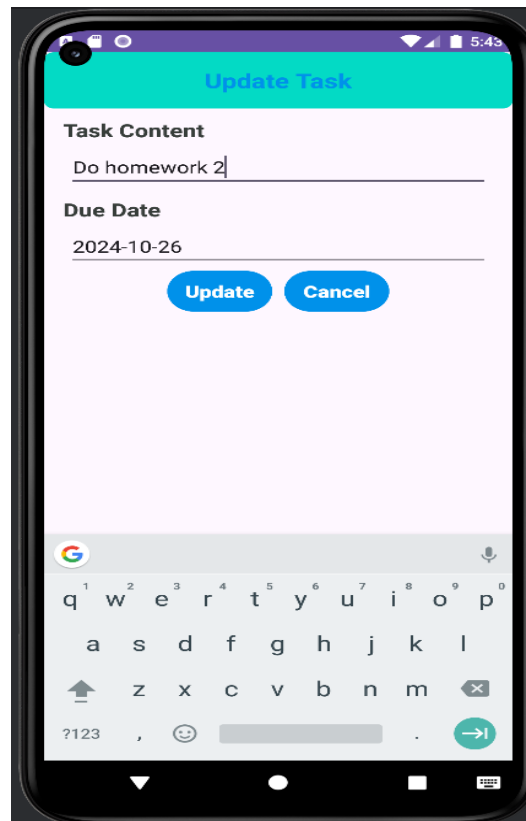
*Figure 8: Added logbook 2 success task*

After saving the first task and clicking on that task to be able to update the task in the update page similar to figure 9



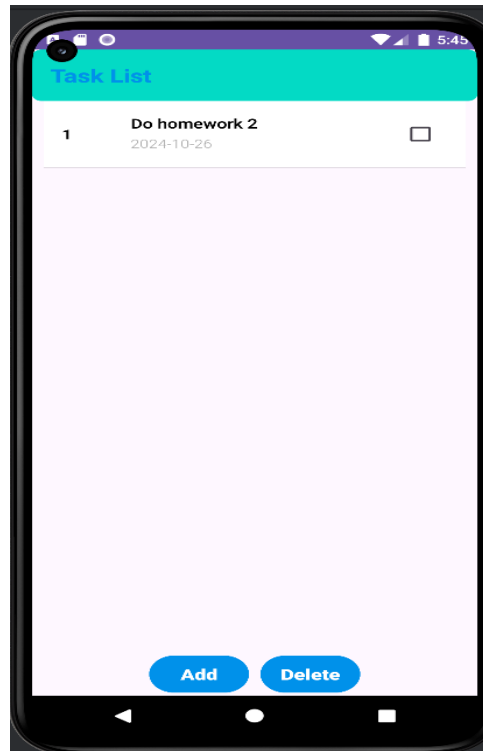
*Figure 9: Updated logbook 2 task interface*

When we go to the update page like figure 10, update the task again.



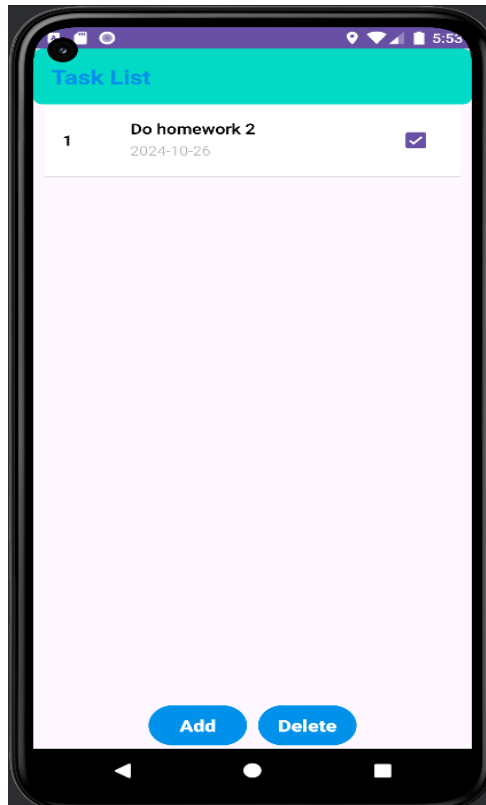
*Figure 10: Fill data in logbook 2 on updae task page*

After updating the information like figure 10, press the save button, the app will notify the update is successful and return to the show list page like figure 11.



*Figure 11: Successfully updated logbook 2 task*

When you want to delete 1 task or many tasks, tick the checkbox on the list page as shown in figure 12.



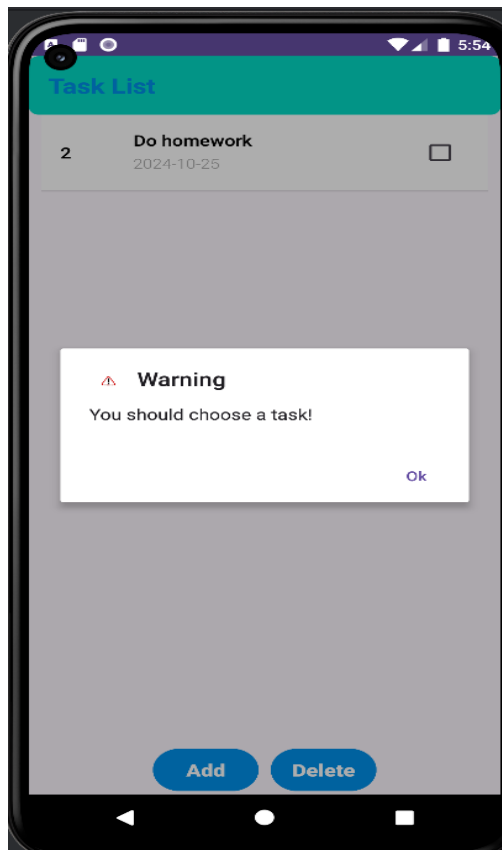
*Figure 12: Select the task to delete from logbook 2*

After selecting the tasks to delete, press the delete button, the task will be deleted and a successful deletion message will appear as shown in figure 13.



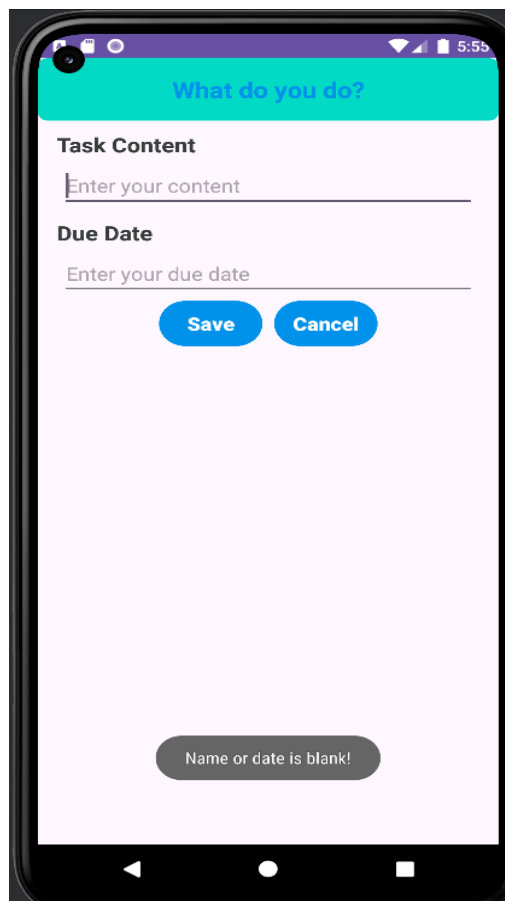
*Figure 13: Successfully deleted logbook 2*

When the user has not selected the task to delete but presses the delete button, the app will display a warning message asking to select a task to delete as shown in figure 14.



*Figure 14: Warning when deleting tasks of logbook 2*

When the user has not filled in all the information and clicks the add or update button, there will be a notification like figure 15.



*Figure 15: Information filling warning of logbook 2*

## 2.2 Code that you wrote

MainActivity: In MainActivity set up a ListView (lvInfos) to display tasks stored in an ArrayList (infos) using a custom adapter (AdapterInfo). Users can add new tasks by clicking the "Add" button, which opens AddActivity. They can also select tasks and delete them using the "Delete" button. If no tasks are selected, a warning dialog will prompt the user to select a task before deleting. The app includes a DrawerLayout for navigation and when the back button is pressed, the app will either close the drawer if it is open or display a confirmation dialog to exit. When returning from AddActivity, new tasks are added to the list and displayed immediately.

```
package com.example.taskmanagementapp;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.view.GravityCompat;
import androidx.drawerlayout.widget.DrawerLayout;
import android.content.DialogInterface;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.ListView;
import android.widget.Toast;

import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {
    public ArrayList<Info> selectedInfos;
    ListView lvInfos;
    ArrayList<Info> infos;
    AdapterInfo adapter;
    Button btnAdd, btnDelete;
    DrawerLayout drawerLayout;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        drawerLayout = findViewById(R.id.drawer_layout);
        btnAdd = findViewById(R.id.btnAdd);
        btnDelete = findViewById(R.id.btnDelete);
        lvInfos=findViewById(R.id.lvInfo);
        infos = new ArrayList<>();
        selectedInfos = new ArrayList<>();
        adapter = new AdapterInfo(this, R.layout.custom_task_list, infos);
```



```

lvInfos.setAdapter(adapter);
btnAdd.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(MainActivity.this, AddActivity.class);
        startActivityForResult(intent , 1);
    }
});
btnDelete.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (selectedInfos == null || selectedInfos.isEmpty()){
            onBackPressed();
        }
        Toast.makeText(MainActivity.this, "Delete successfull!",
Toast.LENGTH_SHORT).show();
        infos.removeAll(selectedInfos);
        selectedInfos.clear();
        adapter.notifyDataSetChanged();
    }
});
}
@Override
public void onBackPressed() {
    if(drawerLayout.isDrawerOpen(GravityCompat.START)){
        drawerLayout.closeDrawer(GravityCompat.START);
    }
    else{
        showExitConfirmationDialog();
    }
}
public void onBackPressed() {
    if(selectedInfos == null || selectedInfos.isEmpty()) {
        showNoTaskSelectedDialog();
    } else {
    }
}
private void showExitConfirmationDialog() {
    AlertDialog.Builder b = new AlertDialog.Builder(MainActivity.this);
    b.setTitle("Question");
    b.setMessage("Are you sure you want to exit?");
    b.setIcon(R.drawable.warning);
    b.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            finish();
        }
    });
}

```

```

        b.setNegativeButton("No", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                dialog.cancel();
            }
        });
        b.create().show();
    }

    private void showNoTaskSelectedDialog() {
        AlertDialog.Builder b = new AlertDialog.Builder(MainActivity.this);
        b.setTitle("Warning");
        b.setMessage("You should choose a task!");
        b.setIcon(R.drawable.warning);
        b.setPositiveButton("Ok", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                dialog.dismiss();
            }
        });
        b.create().show();
    }
}

```

@Override

```

protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == 1 && resultCode == RESULT_OK) {
        if (data != null) {
            String name = data.getStringExtra("name");
            String date = data.getStringExtra("date");
            if (name != null || data!=null) {
                infos.add(new Info(name, date));
                adapter.notifyDataSetChanged();
            }
        }
    }
    if (requestCode == 2 && resultCode == RESULT_OK) {
        if (data != null) {
            String updatedId = data.getStringExtra("updated_id");
            String updatedName = data.getStringExtra("updated_name");
            String updatedDate = data.getStringExtra("updated_date");

            for (int i = 0; i < infos.size(); i++) {
                Info info = infos.get(i);
                if (String.valueOf(info.getId()).equals(updatedId)) {
                    info.setName(updatedName);
                    info.setDate(updatedDate);
                    adapter.notifyDataSetChanged();
                    break;
                }
            }
        }
    }
}

```

```

    }
    }
}
}
}

```

Info class: The Info class represents a task with a unique ID, name, and date. Each Info object is assigned a unique ID when it is created using the static `idCounter` variable, which is incremented each time a new instance is created. The constructor sets the name and date based on the input parameters, and the private static `generateId` method automatically assigns the next available ID. Getter and setter methods allow access to and modification of the id, name, and date for each task.

```

package com.example.taskmanagementapp;

public class Info {
    private static int idCounter = 1;
    private int id;
    private String name;
    private String date;
    public Info(String name, String date) {
        this.id = generateId();
        this.name = name;
        this.date = date;
    }
    private static int generateId() {
        return idCounter++;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDate() {
        return date;
    }
    public void setDate(String date) {
        this.date = date;
    }
}

```

```
}  
}
```

AdapterInfo: To add Info objects from the MainActivity to a list view, the AdapterInfo class extends BaseAdapter. It requires an ArrayList of Info objects, a layout resource, and a context. Each row of the layout is inflated in getView, which retrieves and sets TextView values for the task name (txtTen), date (txtDate), and ID number (txtNumber). The user can choose tasks using the CheckBox (checkBox). The selectedInfos list in MainActivity is updated when the CheckBox is checked or unchecked. Additionally, each row has a clickListener that launches UpdateActivity with the details of the selected task, enabling updates. The list size, item, and item position are provided via the getCount, getItem, and getItemId methods, respectively.

```
package com.example.taskmanagementapp;  
import android.content.Context;  
import android.content.Intent;  
import android.util.Log;  
import android.view.LayoutInflater;  
import android.view.View;  
import android.view.ViewGroup;  
import android.widget.BaseAdapter;  
import android.widget.CheckBox;  
import android.widget.CompoundButton;  
import android.widget.TextView;  
import org.w3c.dom.Text;  
import java.util.ArrayList;  
public class AdapterInfo extends BaseAdapter {  
    private Context context;  
    private int layout;  
    private ArrayList<Info> infos;  
    public AdapterInfo(Context context, int layout, ArrayList<Info> infos) {  
        this.context = context;  
        this.layout = layout;  
        this.infos = infos;  
    }  
  
    @Override  
    public View getView(int i, View view, ViewGroup viewGroup) {  
        //return each line of the view  
        context.getSystemService(context.LAYOUT_INFLATER_SERVICE);  
        LayoutInflater inflater = LayoutInflater.from(context);  
        view=inflater.inflate(layout,viewGroup, false);  
        Info info = infos.get(i);  
        TextView txtTen=view.findViewById(R.id.txtTen);  
        TextView txtDate=view.findViewById(R.id.txtDate);  
        CheckBox checkBox = view.findViewById(R.id.ck);  
        TextView txtNumber = view.findViewById(R.id.txtNumber);
```

```

txtNumber.setText(String.valueOf(info.getId()));
txtTen.setText(infos.get(i).getName());
txtDate.setText(infos.get(i).getDate());
checkBox.setOnCheckedChangeListener(null);
checkBox.setChecked(((MainActivity) context).selectedInfos.contains(info));
checkBox.setOnCheckedChangeListener(new
CompoundButton.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        if (isChecked) {
            ((MainActivity) context).selectedInfos.add(info);
        } else {
            ((MainActivity) context).selectedInfos.remove(info);
        }
    }
});
view.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(context, UpdateActivity.class);
        intent.putExtra("id", String.valueOf(info.getId()));
        intent.putExtra("name", info.getName());
        intent.putExtra("date", info.getDate());
        ((MainActivity) context).startActivityForResult(intent, 2);
    }
});
return view;
}
@Override
public int getCount() {
    return infos.size();
}
@Override
public Object getItem(int position) {
    return infos.get(position);
}
@Override
public long getItemId(int position) {
    return position;
}
}

```

DatePickerFragment class: This class is a DialogFragment that displays a date picker and returns the selected date in the specified format. It takes a TextView or an EditText as the target to display the selected date and a dateFormat string to format the date output.

```

package com.example.taskmanagementapp;
import android.app.DatePickerDialog;
import android.app.Dialog;
import android.os.Bundle;
import android.widget.DatePicker;
import android.widget.EditText;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;
import androidx.lifecycle.viewmodel.CreationExtras;
import java.text.SimpleDateFormat;
import java.util.Calendar;
public class DatePickerFragment extends DialogFragment implements
    DatePickerDialog.OnDateSetListener {

    TextView txtTextViewMapped;
    EditText edtEditTextMapped;
    String dateFormat;
    public DatePickerFragment(TextView txtTextViewMapped, String dateFormat){
        this.txtTextViewMapped=txtTextViewMapped;
        this.dateFormat=dateFormat;
    }
    public DatePickerFragment(EditText edtEditTextMapped, String dateFormat){
        this.edtEditTextMapped=edtEditTextMapped;
        this.dateFormat=dateFormat;
    }
    @NonNull
    @Override
    public Dialog onCreateDialog(@Nullable Bundle savedInstanceState) {
        Calendar calendar=Calendar.getInstance();
        int year=calendar.get(Calendar.YEAR);
        int month = calendar.get(Calendar.MONTH);
        int day = calendar.get(Calendar.DATE);
        return new DatePickerDialog(getActivity(), this, year, month, day);
    }
    @Override
    public void onDateSet(DatePicker datePicker, int i, int i1, int i2) {
        Calendar calendar=Calendar.getInstance();
        calendar.set(i,i1,i2);
        SimpleDateFormat simpleDateFormat=new SimpleDateFormat(dateFormat);
        String sdate=simpleDateFormat.format(calendar.getTime());
        if (txtTextViewMapped!=null){
            txtTextViewMapped.setText(sdate);
        }else if(edtEditTextMapped!=null){
            edtEditTextMapped.setText(sdate);
        }
    }
}

```

```

    @NonNull
    @Override
    public CreationExtras getDefaultViewModelCreationExtras() {
        return super.getDefaultViewModelCreationExtras();
    }
}

```

AddActivity class: The AddActivity class in this code allows users to add a new task by entering a name and selecting a date. When clicking on the date field (edtDate), a date picker dialog appears for easy date selection. If both fields are filled and the user clicks "Save," the task details are passed back to the calling activity with a success message. If the user clicks "Cancel," the activity closes, and a "Cancel successful!" message appears.

```

package com.example.taskmanagementapp;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.DialogFragment;
import java.util.Date;
public class AddActivity extends AppCompatActivity {
    Button btnSave, btnCancel;
    EditText edtName, edtDate;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.add);
        btnCancel = findViewById(R.id.btnCancelAdd);
        btnSave = findViewById(R.id.btnSaveAdd);
        edtName = findViewById(R.id.edtName);
        edtDate = findViewById(R.id.edtDate);
        edtDate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                DialogFragment newFragment = new DatePickerFragment(edtDate, "yyyy-MM-
dd");
                newFragment.show(getSupportFragmentManager(), "datePicker");
            }
        });
        btnSave.setOnClickListener(new View.OnClickListener() {

```

```

@Override
public void onClick(View v) {
    String name = edtName.getText().toString();
    String date = edtDate.getText().toString();
    if(name.isEmpty() || date.isEmpty()){
        Toast.makeText(AddActivity.this, "Name or date is blank!",
Toast.LENGTH_SHORT).show();
    }
    else{
        Intent returnIntent = new Intent();
        returnIntent.putExtra("name", name);
        returnIntent.putExtra("date", date);
        setResult(RESULT_OK, returnIntent);
        Toast.makeText(AddActivity.this, "Add successful!",
Toast.LENGTH_SHORT).show();
        finish();
    }
}
});
btnCancel.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View v) {
    Toast.makeText(AddActivity.this, "Cancel successfull!",
Toast.LENGTH_SHORT).show();
    finish();
}
});
}
}

```

UpdateActivity class: The UpdateActivity class allows users to update a task's name and date. Upon creation, the activity receives the task's ID, name, and date from an intent and displays them in text fields. Users can click on the date field (edtDate) to open a date picker dialog, making it easy to select a new date. If the user updates the name or date and clicks "Update," the new details are passed back to the main activity through an intent, and a "Update successful!" message appears. The "Cancel" button closes the activity with a "Cancel successful!" message, discarding any changes.

```

package com.example.taskmanagementapp;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.DialogFragment;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;

```



```

import android.widget.EditText;
import android.widget.Toast;

public class UpdateActivity extends AppCompatActivity {
    EditText edtName, edtDate;
    Button btnUpdate, btnCancel;
    String taskId;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_update);
        edtName = findViewById(R.id.edtName);
        edtDate = findViewById(R.id.edtDate);
        btnUpdate = findViewById(R.id.btnUpdate);
        btnCancel = findViewById(R.id.btnCancelUpdate);
        Intent intent = getIntent();
        taskId = intent.getStringExtra("id");
        Log.d("taskIdUpdate", taskId);
        String taskName = intent.getStringExtra("name");
        String taskDate = intent.getStringExtra("date");
        edtName.setText(taskName);
        edtDate.setText(taskDate);
        edtDate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                DialogFragment newFragment = new DatePickerFragment(edtDate, "yyyy-MM-
dd");
                newFragment.show(getSupportFragmentManager(), "datePicker");
            }
        });
        btnUpdate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

                String newName = edtName.getText().toString();
                String newDate = edtDate.getText().toString();
                if(newName.isEmpty() || newDate.isEmpty()){
                    Toast.makeText(UpdateActivity.this, "Name or date is blank!",
Toast.LENGTH_SHORT).show();
                }
                else{
                    Intent resultIntent = new Intent();
                    resultIntent.putExtra("updated_id", taskId);
                    resultIntent.putExtra("updated_name", newName);
                    resultIntent.putExtra("updated_date", newDate);
                    setResult(RESULT_OK, resultIntent);
                    Toast.makeText(UpdateActivity.this, "Update successful!",

```

```

Toast.LENGTH_SHORT).show();
        finish();
    }
}
});

btnCancel.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(UpdateActivity.this, "Cancel successfull!",
Toast.LENGTH_SHORT).show();
        finish();
    }
});
}
}

```

activity\_main.xml: This XML code snippet creates the interface for an Android application that uses a DrawerLayout, allowing the user to display a list. Inside the DrawerLayout, there is a LinearLayout containing the title "Task List", a ListView to display the list of tasks, and another LinearLayout containing two buttons "Add" and "Delete". This structure makes it easy for the user to manage tasks by adding or removing items from the list, while taking advantage of the flexible space of the ListView.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Main content layout -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        tools:context=".MainActivity">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Task List"
            style="@style/TextViewHeading"
            android:textStyle="bold" />

        <ListView
            android:id="@+id/lvInfo"

```

```
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_marginHorizontal="10dp"
    android:layout_weight="1" /> <!-- Use weight for flexibility -->
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal">
```

```
    <Button
        android:id="@+id/btnAdd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="10dp"
        style="@style/CustomButtonStyle"
        android:text="Add" />
```

```
    <Button
        android:id="@+id/btnDelete"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/CustomButtonStyle"
        android:text="Delete" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```

```
</androidx.drawerlayout.widget.DrawerLayout>
```

activity\_main.xml: This XML snippet describes the interface for an UpdateActivity in a task manager application. The interface includes a title "Update Task," two input fields (EditText) for the task content and due date, and two buttons for the user to update the information or cancel the action. All of the components are arranged vertically in a LinearLayout, which creates a clean and easy-to-use interface.

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".UpdateActivity">
```

```
    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Update Task"
```

```
        android:textAlignment="center"
        style="@style/TextViewHeading"
    />
```

```
<TextView
    android:id="@+id/textView2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    style="@style/TextViewLabel"
    android:text="Task Content" />
```

```
<EditText
    android:id="@+id/edtName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:layout_marginEnd="20sp"
    android:layout_marginStart="20sp"
    android:hint="Enter your content" />
```

```
<TextView
    android:id="@+id/textView3"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    style="@style/TextViewLabel"
    android:text="Due Date" />
```

```
<EditText
    android:id="@+id/edtDate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginEnd="20sp"
    android:layout_marginStart="20sp"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter your due date" />
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal">
```

```
    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
style="@style/CustomButtonStyle"
android:layout_marginEnd="10dp"
android:text="Update" />
```

```
<Button
    android:id="@+id/btnCancelUpdate"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/CustomButtonStyle"
    android:text="Cancel" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```

add.xml: This XML code describes the interface for AddActivity in a task management application, including a title, label, and input fields for the task content and due date. The interface uses a LinearLayout with a vertical orientation, which helps organize the components logically. The components include a TextView that displays the question "What do you do?", two EditTexts that allow the user to enter the task content and due date, and two "Save" and "Cancel" buttons at the bottom to perform or cancel the task addition. This design is aimed at simplicity and convenience for users during the input process.

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    tools:context=".AddActivity">
```

```
<TextView
```

```
    android:id="@+id/textView"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="What do you do?"
```

```
    android:textAlignment="center"
```

```
    style="@style/TextViewHeading"
```

```
/>
```

```
<TextView
```

```
    android:id="@+id/textView2"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    style="@style/TextViewLabel"
```

```
    android:text="Task Content" />
```

```
<EditText
```

```
    android:id="@+id/edtName"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
android:ems="10"
android:inputType="text"
android:layout_marginEnd="20sp"
android:layout_marginStart="20sp"
android:hint="Enter your content"
/>
```

```
<TextView
    android:id="@+id/textView3"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    style="@style/TextViewLabel"
    android:text="Due Date" />
```

```
<EditText
    android:id="@+id/edtDate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginEnd="20sp"
    android:layout_marginStart="20sp"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter your due date" />
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal">
```

```
<Button
    android:id="@+id/btnSaveAdd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/CustomButtonStyle"
    android:layout_marginEnd="10dp"
    android:text="Save" />
```

```
<Button
    android:id="@+id/btnCancelAdd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    style="@style/CustomButtonStyle"
    android:text="Cancel" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```

custom\_task\_list.xml: This XML code defines the interface for a task list item in a task manager application. The interface consists of a main LinearLayout with a white background and padding around it, which contains a sub-LinearLayout with a horizontal layout. In the sub-LinearLayout, there is a TextView that displays the task number, followed by a LinearLayout that contains two TextViews: one for the task name and one for the due date. Finally, there is a CheckBox that allows the user to mark the task as complete or delete it. This design is not only neat, but also ensures that the user can easily interact with and track the progress of the tasks.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="8dp"
    android:background="@android:color/white"
    >

    <!-- Main container -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="8dp"
        android:elevation="4dp"
        android:weightSum="6">

        <!-- Order Number TextView -->
        <TextView
            android:id="@+id/txtNumber"
            android:layout_width="0dp"
            android:layout_weight="1"
            android:layout_height="wrap_content"
            android:text="1."
            android:textSize="16sp"
            android:textColor="@android:color/black"
            android:layout_gravity="center_vertical"
            android:paddingEnd="12dp"
            android:fontFamily="sans-serif-medium" />

        <!-- Text container -->
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:paddingStart="12dp"
            android:layout_weight="5">

            <!-- Name TextView -->
```

```

<TextView
    android:id="@+id/txtTen"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Content"
    android:textSize="16sp"
    android:textColor="@android:color/black"
    android:fontFamily="sans-serif-medium" />

<!-- Date TextView -->
<TextView
    android:id="@+id/txtDate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Date"
    android:textSize="14sp"
    android:textColor="@android:color/darker_gray"
    android:fontFamily="sans-serif-light"
    android:paddingTop="4dp" />
</LinearLayout>

```

```

<!-- Delete CheckBox -->
<CheckBox
    android:id="@+id/ck"
    android:layout_width="wrap_content"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:layout_gravity="center_vertical"
    />
</LinearLayout>
</LinearLayout>

```

colors.xml: The above XML code defines a set of color resources for the Android application. Each color is assigned a unique name along with its corresponding HEX color code.

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>
    <color name="red">#D50000</color>
    <color name="colorPrimary">#0091EA</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="colorLable">#373D3A</color>
</resources>

```



strings.xml: This strings.xml file contains resource strings for Android applications.

```
<resources>
    <string name="app_name">Task Management App</string>
</resources>
```

themes.xml:

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.TaskManagementApp"
parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.TaskManagementApp"
parent="Base.Theme.TaskManagementApp" />
    <style name="TextViewHeading">
        <item name="android:textColor">@color/colorPrimary</item> <!-- Change to your
desired color -->
        <item name="android:textSize">22sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:padding">16dp</item> <!-- Add padding for better spacing -->
        <item name="android:background">@drawable/textview_background</item> <!--
Optional: custom background -->
    </style>
```

```
    <style name="TextViewLabel">
        <item name="android:textColor">@color/colorLable</item> <!-- Change to your desired
color -->
        <item name="android:textSize">20sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:paddingHorizontal">16dp</item> <!-- Add padding for better
spacing -->
        <item name="android:paddingTop">10dp</item>
        <item name="android:paddingBottom">5dp</item>

    </style>
```

```
    <style name="CustomButtonStyle">
        <item name="android:backgroundTint">@color/colorPrimary</item>
        <item name="android:textColor">@android:color/white</item> <!-- White text for
contrast -->
        <item name="android:textSize">18sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:padding">10dp</item>
```

</style>  
</resources>

### Logbook 3

#### 1. Basic Information

|                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1 Student name                                                                                                                                                                      | Vo Tan Nghia                                                                                                                                                                                                                                                                                                                                |
| 1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work. | Do it alone                                                                                                                                                                                                                                                                                                                                 |
| 1.3 Which Exercise is this? Tick as appropriate.                                                                                                                                      | <ul style="list-style-type: none"><li>• Exercise 1 <input type="checkbox"/></li><li>• Exercise 2 <input type="checkbox"/></li><li>• Exercise 3 <input checked="" type="checkbox"/></li></ul>                                                                                                                                                |
| 1.4 How well did you complete the exercise? Tick as appropriate.                                                                                                                      | <ul style="list-style-type: none"><li>• I tried but couldn't complete it <input type="checkbox"/></li><li>• I did it but I feel I should have done better <input type="checkbox"/></li><li>• I did everything that was asked <input type="checkbox"/></li><li>• I did more than was asked for <input checked="" type="checkbox"/></li></ul> |
| 1.5 Briefly explain your answer to question 1.4<br><br>Without any explanation/justification, your scores will be deducted.                                                           | In logbook 3, I completed the assignment requirements and added a dialog for the user to add a date. Additionally, I did some basic debugging for the functions.                                                                                                                                                                            |

## 2. Exercise answer

### 2.1 Screen shots demonstrating what you achieved

Figure 16 is the task list page interface of logbook 3

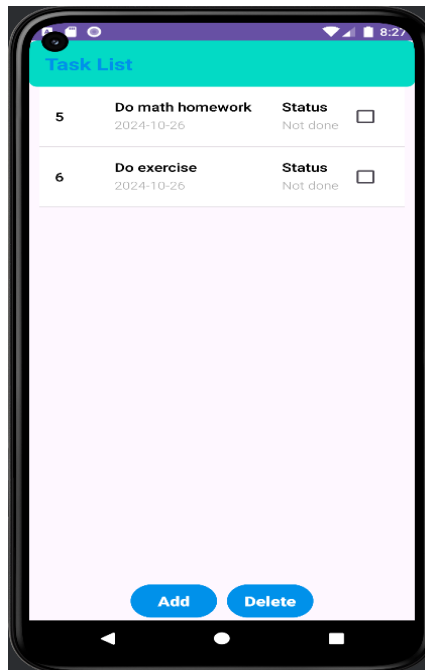


Figure 16: Task list logbook 3 interface

Figure 17 is the interface of the add list page. The page has a place to fill in the task content, date and status.

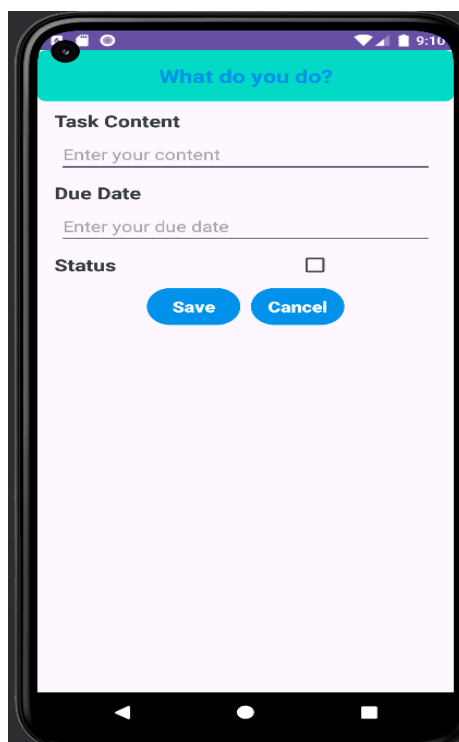


Figure 17: Logbook 3 task add page

Fill in the required information in the add list page such as content and date as figure 18

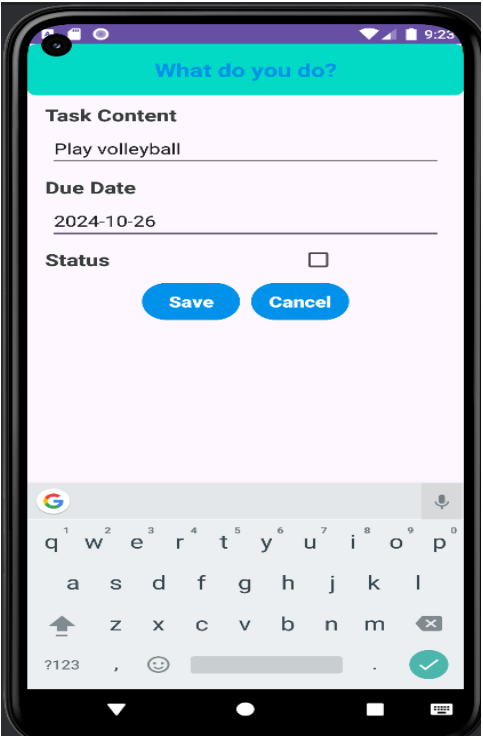


Figure 18: Fill data in add page in logbook 3

After filling in all the information and pressing the save button, it will display a successful add message and return to the page showing the task list as shown in figure 19.

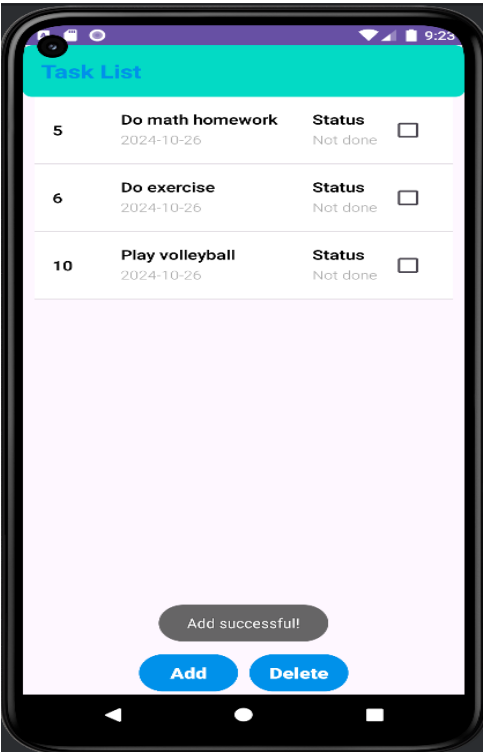


Figure 19: Added logbook 3 success task

Figure 20 is the task update page when clicking on task number 10 on the show list page

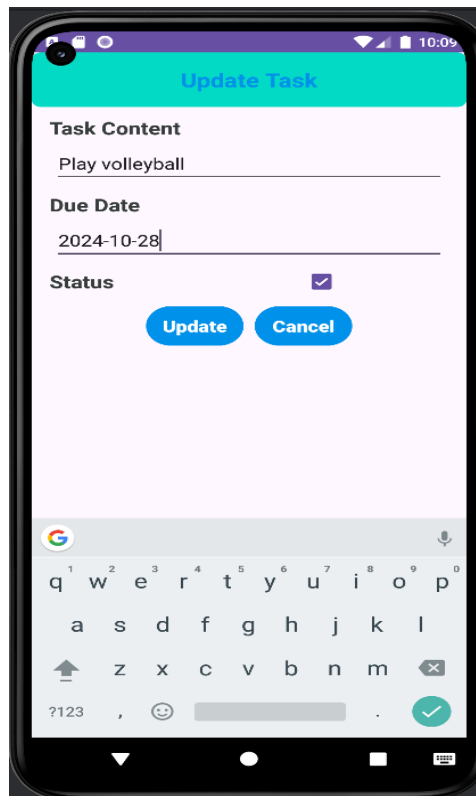


Figure 20: Updated logbook 3 task interface

After updating and pressing the save button, it will display a successful update message and display the show list page again.

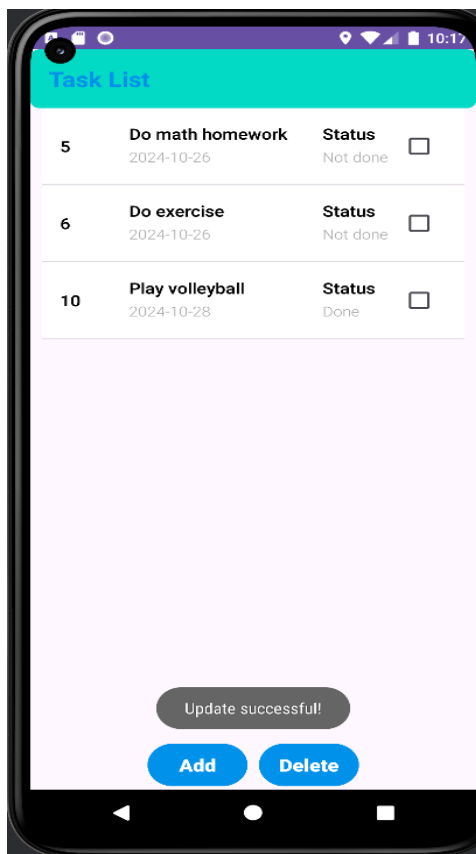
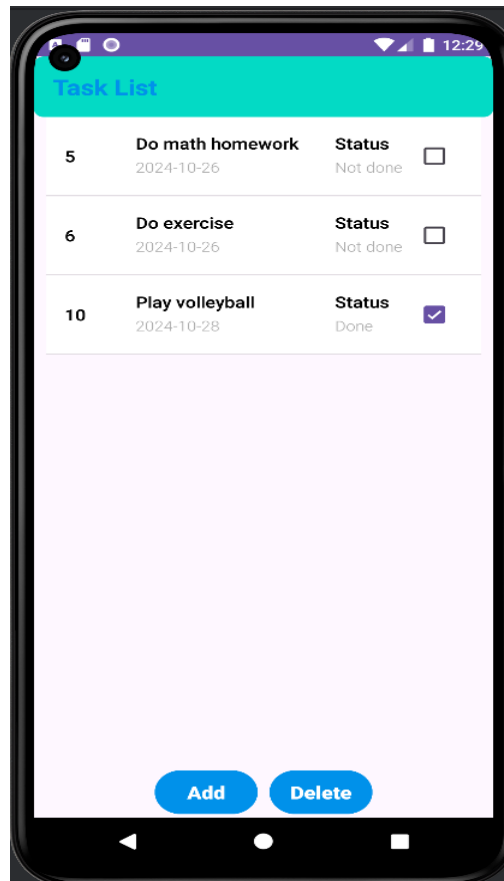


Figure 21: Successfully updated logbook 3 task

When you want to delete a task on the show list page, tick the task box to delete as shown in figure 22.



*Figure 22: Select the task to delete from logbook 3*

When you select the task to delete in figure 22, press the delete button below, it will display a successful deletion message and delete the task as in figure 23.

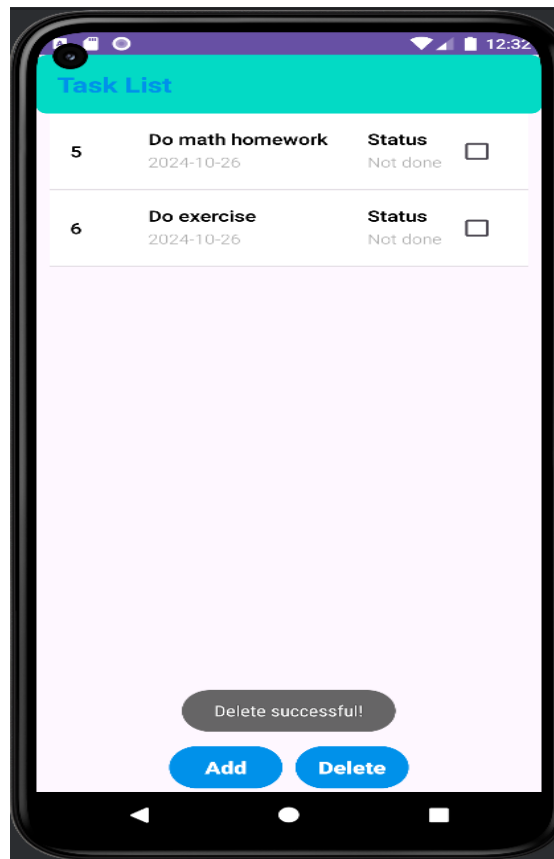


Figure 23: Successfully deleted logbook 3

In figure 24 interface, when the user has not selected any task to delete and clicks delete, a message will appear asking to select a task to delete.

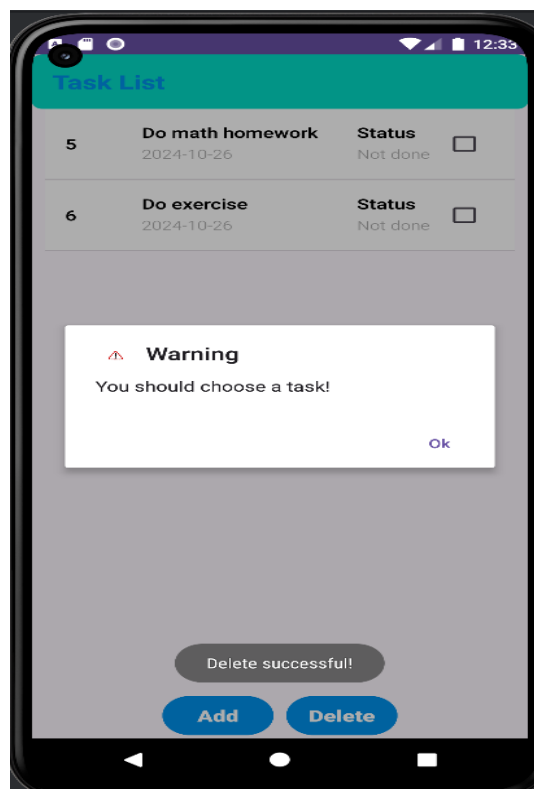


Figure 24: Warning when deleting tasks of logbook 3

In figure 25 interface, when the user has not filled in all the information when adding or updating data, a reminder message will appear.

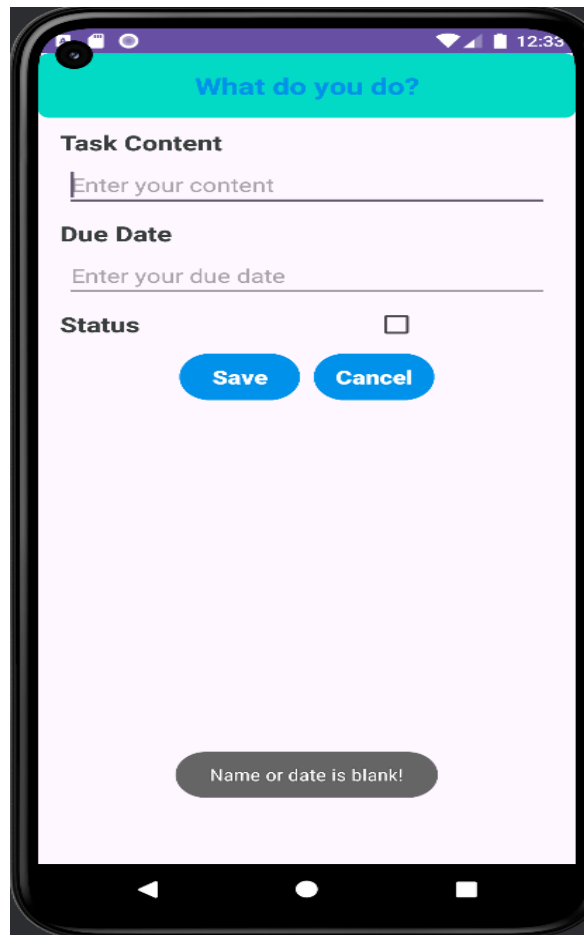


Figure 25: Information filling warning of logbook 3

## 2.2 Code that you wrote

MainActivity class: The MainActivity class in the code above manages the main activity of the application, displaying the list of tasks and allowing the user to add or remove tasks. In the onCreate method, the interface components such as ListView, Button, and DrawerLayout are set up and connected to the elements in the interface XML. The task data from the SQLite database is retrieved and put into the infos list, which is then displayed in the ListView using an Adapter. The btnAdd button is configured to go to AddActivity, where the user can add new tasks. The btnDelete button allows the selected tasks to be removed from the list and database. Additionally, the onBackPressed method handles the event when the user presses the back button, displays a confirmation dialog to exit the application if the menu is not open, and a warning dialog when no tasks are selected to be deleted.

```
package com.example.taskmanagementapp;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.view.GravityCompat;
import androidx.drawerlayout.widget.DrawerLayout;
import android.content.DialogInterface;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
```



```

import android.widget.Button;
import android.widget.ListView;
import android.widget.Toast;
import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {
    public ArrayList<Info> selectedInfos;
    ListView lvInfos;
    ArrayList<Info> infos;
    AdapterInfo adapter;
    Button btnAdd, btnDelete;
    Database database;
    DrawerLayout drawerLayout;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        drawerLayout = findViewById(R.id.drawer_layout);
        btnAdd = findViewById(R.id.btnAdd);
        btnDelete = findViewById(R.id.btnDelete);
        lvInfos=findViewById(R.id.lvInfo);
        infos = new ArrayList<>();
        selectedInfos = new ArrayList<>();
        database = new Database(this, "Task", null, 1);
        String ex ="Create table if not Exists Task(ID integer primary key autoincrement, " +
            "Name varchar(200), " +
            "Date varchar(200)," +
            "Status BOOLEAN)";

        database.Execute(ex);
        Cursor dataCursor = database.GetData("SELECT * FROM Task");
        while (dataCursor.moveToNext()) {
            String name = dataCursor.getString(1);
            String date = dataCursor.getString(2);
            String id = dataCursor.getString(0);
            int statusInt = dataCursor.getInt(3);
            Boolean statusCheck = (statusInt == 1);
            infos.add(new Info(id, name, date, statusCheck));
        }
        dataCursor.close();
        adapter = new AdapterInfo(this, R.layout.custom_task_list, infos);
        lvInfos.setAdapter(adapter);
        btnAdd.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(MainActivity.this, AddActivity.class);
                startActivityForResult(intent , 1);
            }
        }
    }

```

```

});
btnDelete.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (selectedInfos == null || selectedInfos.isEmpty()){
            onBackPressed();
        }
        for (Info info : selectedInfos) {
            String deleteQuery = "DELETE FROM Task WHERE ID = " + info.getId();
            database.Execute(deleteQuery);
        }
        Toast.makeText(MainActivity.this, "Delete successful!",
Toast.LENGTH_SHORT).show();
        infos.removeAll(selectedInfos);
        selectedInfos.clear();
        adapter.notifyDataSetChanged();
    }
});
}
@Override
public void onBackPressed() {
    if(drawerLayout.isDrawerOpen(GravityCompat.START)){
        drawerLayout.closeDrawer(GravityCompat.START);
    }
    else{
        showExitConfirmationDialog();
    }
}
public void onBackPressed() {
    if(selectedInfos == null || selectedInfos.isEmpty()) {
        showNoTaskSelectedDialog();
    }
}
private void showExitConfirmationDialog() {
    AlertDialog.Builder b = new AlertDialog.Builder(MainActivity.this);
    b.setTitle("Question");
    b.setMessage("Are you sure you want to exit?");
    b.setIcon(R.drawable.warning);
    b.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            finish();
        }
    });
    b.setNegativeButton("No", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            dialog.cancel();
        }
    }

```

```

});
b.create().show();
}

```

```

private void showNoTaskSelectedDialog() {
    AlertDialog.Builder b = new AlertDialog.Builder(MainActivity.this);
    b.setTitle("Warning");
    b.setMessage("You should choose a task!");
    b.setIcon(R.drawable.warning);
    b.setPositiveButton("Ok", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            dialog.dismiss();
        }
    });
    b.create().show();
}

```

```

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == 1 && resultCode == RESULT_OK) {
        if (data != null) {
            String name = data.getStringExtra("name");
            String date = data.getStringExtra("date");
            String id = data.getStringExtra("id");
            boolean status = data.getBooleanExtra("status", false);
            if (name != null || data!=null) {
                infos.add(new Info(id,name, date, status));
                adapter.notifyDataSetChanged();
            }
        }
    }
    if (requestCode == 2 && resultCode == RESULT_OK) {
        if (data != null) {
            String updatedId = data.getStringExtra("updated_id");
            String updatedName = data.getStringExtra("updated_name");
            String updatedDate = data.getStringExtra("updated_date");
            boolean updatedStatus = data.getBooleanExtra("updated_status", false);
            for (int i = 0; i < infos.size(); i++) {
                Info info = infos.get(i);
                if (info.getId().equals(updatedId)) {
                    info.setName(updatedName);
                    info.setDate(updatedDate);
                    info.setStatus(updatedStatus);
                    adapter.notifyDataSetChanged();
                    break;
                }
            }
        }
    }
}

```

```

    }
}
}
}
}

```

Info class: The Info class represents information about a task in a task management application, consisting of four properties: id (the task ID), name (the task name), date (the task date), and status (the task completion status). The class provides getter and setter methods to access and change the properties, and a constructor to initialize the object with an initial value for each property.

```
package com.example.taskmanagementapp;
```

```

public class Info {
    private String id;
    private String name;
    private String date;
    private Boolean status;

    public Boolean getStatus() {
        return status;
    }
    public void setStatus(Boolean status) {
        this.status = status;
    }
    public Info(String id, String name, String date, Boolean status) {
        this.id = id;
        this.name = name;
        this.date = date;
        this.status = status;
    }
    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDate() {
        return date;
    }
    public void setDate(String date) {

```

```

        this.date = date;    }
    }

```

DatePickerFragment class: The DatePickerFragment class is a DialogFragment that shows a date picker and gives back the date that was chosen in the format that was given. It requires a dateFormat string to format the date output and a TextView or EditText as the target to display the chosen date.

```

package com.example.taskmanagementapp;
import android.app.DatePickerDialog;
import android.app.Dialog;
import android.os.Bundle;
import android.widget.DatePicker;
import android.widget.EditText;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;
import androidx.lifecycle.viewmodel.CreationExtras;
import java.text.SimpleDateFormat;
import java.util.Calendar;
public class DatePickerFragment extends DialogFragment implements
    DatePickerDialog.OnDateSetListener {

    TextView txtTextViewMapped;
    EditText edtEditTextMapped;
    String dateFormat;
    public DatePickerFragment(TextView txtTextViewMapped, String dateFormat){
        this.txtTextViewMapped=txtTextViewMapped;
        this.dateFormat=dateFormat;
    }
    public DatePickerFragment(EditText edtEditTextMapped, String dateFormat){
        this.edtEditTextMapped=edtEditTextMapped;
        this.dateFormat=dateFormat;
    }
    @NonNull
    @Override
    public Dialog onCreateDialog(@Nullable Bundle savedInstanceState) {
        Calendar calendar=Calendar.getInstance();
        int year=calendar.get(Calendar.YEAR);
        int month = calendar.get(Calendar.MONTH);
        int day = calendar.get(Calendar.DATE);
        return new DatePickerDialog(getActivity(), this, year, month, day);
    }
    @Override
    public void onDateSet(DatePicker datePicker, int i, int i1, int i2) {
        Calendar calendar=Calendar.getInstance();

```

```

        calendar.set(i,i1,i2);
        SimpleDateFormat simpleDateFormat=new SimpleDateFormat(dateFormat);
        String sdate=simpleDateFormat.format(calendar.getTime());
        if (txtTextViewMapped!=null){
            txtTextViewMapped.setText(sdate);
        }else if(edtEditTextMapped!=null){
            edtEditTextMapped.setText(sdate);
        }
    }
}
@NonNull
@Override
public CreationExtras getDefaultViewModelCreationExtras() {
    return super.getDefaultViewModelCreationExtras();
}
}

```

AdapterInfo class: The AdapterInfo class is a custom adapter that displays a list of Info in a ListView. In the getView method, it uses a LayoutInflater to create a stream view for each item, displaying information such as id, name, date, and completion status. The CheckBox allows the user to select items, and the state of the CheckBox updates the selectedInfos in the MainActivity. When an item is clicked, the adapter opens an UpdateActivity so the user can update the information of the selected item.

```

package com.example.taskmanagementapp;
import android.content.Context;
import android.content.Intent;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.CheckBox;
import android.widget.CompoundButton;
import android.widget.TextView;
import org.w3c.dom.Text;
import java.util.ArrayList;
public class AdapterInfo extends BaseAdapter {
    private Context context;
    private int layout;
    private ArrayList<Info> infos;
    public AdapterInfo(Context context, int layout, ArrayList<Info> infos) {
        this.context = context;
        this.layout = layout;
        this.infos = infos;
    }
    @Override
    public View getView(int i, View view, ViewGroup viewGroup) {
        //return each line of the view
        context.getSystemService(context.LAYOUT_INFLATER_SERVICE);
        LayoutInflater inflater = LayoutInflater.from(context);
    }
}

```

```

view=inflater.inflate(layout,viewGroup, false);

Info info = infos.get(i);
TextView txtTen=view.findViewById(R.id.txtTen);
TextView txtPhone=view.findViewById(R.id.txtDate);
CheckBox checkBox = view.findViewById(R.id.ck);
TextView txtNumber = view.findViewById(R.id.txtNumber);
TextView txtStatus = view.findViewById(R.id.txtStatus);
txtNumber.setText(info.getId());
txtTen.setText(infos.get(i).getName());
txtPhone.setText(infos.get(i).getDate());
if(infos.get(i).getStatus()==true){
    txtStatus.setText("Done");
}
else {
    txtStatus.setText("Not done");
}
checkBox.setOnCheckedChangeListener(null);
checkBox.setChecked(((MainActivity) context).selectedInfos.contains(info));
checkBox.setOnCheckedChangeListener(new
CompoundButton.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        if (isChecked) {
            ((MainActivity) context).selectedInfos.add(info);
        } else {
            ((MainActivity) context).selectedInfos.remove(info);
        }
    }
});
view.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(context, UpdateActivity.class);
        intent.putExtra("id", info.getId());
        intent.putExtra("name", info.getName());
        intent.putExtra("date", info.getDate());
        intent.putExtra("status", info.getStatus());
        ((MainActivity) context).startActivityForResult(intent, 2); // 2 là request code
    }
});
return view;
}
@Override
public int getCount() {
    return infos.size();
}

```

```

@Override
public Object getItem(int position) {
    return infos.get(position);
}

@Override
public long getItemId(int position) {
    return position;
}
}

```

DatePickerFragment class: The DatePickerFragment class is a DialogFragment that helps the user select a date from the DatePickerDialog dialog. When created, this class can be associated with a TextView or EditText to display the selected date. In onCreateDialog, the class uses the current date as the default date in the DatePicker. After the user selects a date, the onDateSet method formats the date according to the pattern (dateFormat) and displays the result in the associated TextView or EditText.

```

package com.example.taskmanagementapp;
import android.app.DatePickerDialog;
import android.app.Dialog;
import android.os.Bundle;
import android.widget.DatePicker;
import android.widget.EditText;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.DialogFragment;
import androidx.lifecycle.viewmodel.CreationExtras;

import java.text.SimpleDateFormat;
import java.util.Calendar;

public class DatePickerFragment extends DialogFragment implements
    DatePickerDialog.OnDateSetListener {

    TextView txtTextViewMapped;
    EditText edtEditTextMapped;
    String dateFormat;
    public DatePickerFragment(TextView txtTextViewMapped, String dateFormat){
        this.txtTextViewMapped=txtTextViewMapped;
        this.dateFormat=dateFormat;
    }
    public DatePickerFragment(EditText edtEditTextMapped, String dateFormat){
        this.edtEditTextMapped=edtEditTextMapped;
        this.dateFormat=dateFormat;
    }
}

```



```

@NonNull
@Override
public Dialog onCreateDialog(@Nullable Bundle savedInstanceState) {
    Calendar calendar=Calendar.getInstance();
    int year=calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH);
    int day = calendar.get(Calendar.DATE);
    return new DatePickerDialog(getActivity(), this, year, month, day);
}

@Override
public void onDateSet(DatePicker datePicker, int i, int i1, int i2) {
    Calendar calendar=Calendar.getInstance();
    calendar.set(i,i1,i2);
    SimpleDateFormat simpleDateFormat=new SimpleDateFormat(dateFormat);
    String sdate=simpleDateFormat.format(calendar.getTime());
    if (txtTextViewMapped!=null){
        txtTextViewMapped.setText(sdate);
    }else if(edtEditTextMapped!=null){
        edtEditTextMapped.setText(sdate);
    }
}

@NonNull
@Override
public CreationExtras getDefaultViewModelCreationExtras() {
    return super.getDefaultViewModelCreationExtras();
}
}

```

Database class: The Database class extends SQLiteOpenHelper to manage SQLite databases in the application. The Execute method executes queries that do not return results (such as CREATE, INSERT, UPDATE, DELETE), while Getdata returns a Cursor to retrieve results from SELECT queries. The onCreate and onUpgrade methods can be overridden to create tables and handle database upgrades as needed.

```

package com.example.taskmanagementapp;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import androidx.annotation.Nullable;
public class Database extends SQLiteOpenHelper {
    public Database(@Nullable Context context, @Nullable String name, @Nullable
SQLiteDatabase.CursorFactory factory, int version) {
        super(context, name, factory, version);
    }
}

```

```

public void Execute(String query) {
    SQLiteDatabase database = getWritableDatabase();
    database.execSQL(query);
}
public Cursor Getdata(String query) {
    SQLiteDatabase database = getReadableDatabase();
    return database.rawQuery(query, null);
}
@Override
public void onCreate(SQLiteDatabase db) {
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
}
}

```

AddActivity class: The AddActivity class is responsible for adding a new task to the database in the task manager application. In the onCreate method, it sets up the user interface and the function buttons, including btnSave to save the task and btnCancel to cancel the operation. When the user presses the "Save" button, the application checks if the name and date fields are empty. If not, it executes an SQL query to add the new task to the Task table. After adding, it retrieves the ID of the newly inserted task and returns the data to MainActivity via an Intent. The "Cancel" button closes AddActivity without saving the changes.

```

package com.example.taskmanagementapp;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.DialogFragment;
import java.util.Date;
public class AddActivity extends AppCompatActivity {
    Button btnSave, btnCancel;
    EditText edtName, edtDate;
    CheckBox cbStatus;
    Database database;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.add);
        btnCancel = findViewById(R.id.btnCancelAdd);
        btnSave = findViewById(R.id.btnSaveAdd);
        cbStatus = findViewById(R.id.cbStatus);
    }
}

```

```

edtName = findViewById(R.id.edtName);
edtDate = findViewById(R.id.edtDate);
edtDate.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        DialogFragment newFragment = new DatePickerFragment(edtDate, "yyyy-MM-
dd");
        newFragment.show(getSupportFragmentManager(), "datePicker");
    }
});
btnSave.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String name = edtName.getText().toString();
        String date = edtDate.getText().toString();
        boolean status = cbStatus.isChecked();
        if(name.isEmpty() || date.isEmpty()){
            Toast.makeText(AddActivity.this, "Name or date is blank!",
Toast.LENGTH_SHORT).show();
        }
        else{
            database = new Database(AddActivity.this, "Task", null, 1);
            String insertQuery = "INSERT INTO Task (Name, Date, Status) VALUES ('" +
name + "', '" + date + "', '" + status + "')";
            database.Execute(insertQuery);
            String lastIdQuery = "SELECT last_insert_rowid()";
            Cursor cursor = database.Getdata(lastIdQuery);
            cursor.moveToFirst();
            String id = cursor.getString(0);
            cursor.close();
            Intent returnIntent = new Intent();
            returnIntent.putExtra("id", id);
            returnIntent.putExtra("name", name);
            returnIntent.putExtra("date", date);
            returnIntent.putExtra("status", status);
            setResult(RESULT_OK, returnIntent);
            Toast.makeText(AddActivity.this, "Add successful!",
Toast.LENGTH_SHORT).show();
            finish();
        }
    }
});
btnCancel.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(AddActivity.this, "Cancel successful!",
Toast.LENGTH_SHORT).show();
        finish();
    }
});

```

```

    }
    });
}
}

```

UpdateActivity class: The UpdateActivity class in the task management application allows the user to update information about an existing task. When initialized, the class receives data from the Intent such as the task ID, name, date, and status and displays them in the interface. When the user clicks the "Update" button, the application checks the validity of the input fields and executes an SQL UPDATE query to update the information in the database. If the update is successful, the class returns the updated information to the previous activity. If the user selects "Cancel", the class closes without making any changes, making it easy for the user to manage and edit their tasks.

```

package com.example.taskmanagementapp;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Toast;
public class UpdateActivity extends AppCompatActivity {
    EditText edtName, edtDate;
    Button btnUpdate, btnCancel;
    CheckBox cbStatus;
    Database database;
    String taskId;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_update);
        edtName = findViewById(R.id.edtName);
        edtDate = findViewById(R.id.edtDate);
        btnUpdate = findViewById(R.id.btnUpdate);
        btnCancel = findViewById(R.id.btnCancelUpdate);
        cbStatus = findViewById(R.id.cbStatus);
        Intent intent = getIntent();
        taskId = intent.getStringExtra("id");
        String taskName = intent.getStringExtra("name");
        String taskDate = intent.getStringExtra("date");
        Boolean taskStatus = intent.getBooleanExtra("status", false);
        edtName.setText(taskName);
        edtDate.setText(taskDate);
        cbStatus.setChecked(taskStatus);

        database = new Database(this, "Task", null, 1);
        btnUpdate.setOnClickListener(new View.OnClickListener() {

```

```

@Override
public void onClick(View v) {
    String newName = edtName.getText().toString();
    String newDate = edtDate.getText().toString();
    boolean newStatus = cbStatus.isChecked();
    if(newName.isEmpty() || newDate.isEmpty()){
        Toast.makeText(UpdateActivity.this, "Name or date is blank!",
Toast.LENGTH_SHORT).show();
    }
    else{
        String updateQuery = "UPDATE Task SET Name = " + newName + ", Date = "
+ newDate + ", Status = " + (newStatus ? 1 : 0) + " WHERE ID = " + taskId;
        database.Execute(updateQuery);
        Intent resultIntent = new Intent();
        resultIntent.putExtra("updated_id", taskId);
        resultIntent.putExtra("updated_name", newName);
        resultIntent.putExtra("updated_date", newDate);
        resultIntent.putExtra("updated_status", newStatus);
        Toast.makeText(UpdateActivity.this, "Update successful!",
Toast.LENGTH_SHORT).show();
        setResult(RESULT_OK, resultIntent);
        finish();
    }
}
});

btnCancel.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View v) {
    Toast.makeText(UpdateActivity.this, "Cancel successful!",
Toast.LENGTH_SHORT).show();
    finish();
}
});
}
}

```

activity\_main.xml: This XML code defines the user interface for an Android application that uses a DrawerLayout, which allows for a sliding task list. Inside the DrawerLayout, there is a LinearLayout containing the title "Task List" and a ListView that displays the list of tasks. Below the list, there are two buttons "Add" and "Delete" so the user can add or remove tasks.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

```

```

<!-- Main content layout -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Task List"
        style="@style/TextViewHeading"
        android:textStyle="bold" />

    <ListView
        android:id="@+id/lvInfo"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_marginHorizontal="10dp"
        android:layout_weight="1" /> <!-- Use weight for flexibility -->

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_horizontal">

        <Button
            android:id="@+id/btnAdd"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="10dp"
            style="@style/CustomButtonStyle"
            android:text="Add" />

        <Button
            android:id="@+id/btnDelete"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            style="@style/CustomButtonStyle"
            android:text="Delete" />
    </LinearLayout>
</LinearLayout>

</androidx.drawerlayout.widget.DrawerLayout>

```

Activity\_update.xml: The XML code above defines the interface for the task update activity in an Android application. The interface consists of a main LinearLayout set up vertically,

containing components such as the "Update Task" title, labels and input fields for the task's content and due date, along with a CheckBox to mark the completion status. Finally, there are two "Update" and "Cancel" buttons that allow the user to perform the update or cancel the action. The interface is designed to be intuitive and easy to use, allowing users to quickly update task information.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".UpdateActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Update Task"
        android:textAlignment="center"
        style="@style/TextViewHeading"
    />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        style="@style/TextViewLabel"
        android:text="Task Content" />

    <EditText
        android:id="@+id/edtName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="text"
        android:layout_marginEnd="20sp"
        android:layout_marginStart="20sp"
        android:hint="Enter your content" />

    <TextView
        android:id="@+id/textView3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        style="@style/TextViewLabel"
        android:text="Due Date" />

    <EditText
        android:id="@+id/edtDate"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginEnd="20sp"
        android:layout_marginStart="20sp"
        android:ems="10"
        android:inputType="text"
        android:hint="Enter your due date" />
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal"
    android:weightSum="3">

    <TextView
        android:id="@+id/textView4"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        style="@style/TextViewLabel"
        android:layout_weight="1"
        android:text="Status" />
    <CheckBox
        android:id="@+id/cbStatus"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:layout_gravity="center_vertical"
        />
</LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal">

    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/CustomButtonStyle"
        android:layout_marginEnd="10dp"
        android:text="Update" />

    <Button
        android:id="@+id/btnCancelUpdate"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/CustomButtonStyle"

```



```

        android:text="Cancel" />
    </LinearLayout>
</LinearLayout>

```

add.xml: The above XML code defines the interface for an "Add Task" activity in an Android application. The interface is organized in a vertical LinearLayout, including components such as: a central title "What do you do?" to guide the user, input fields for the content and due date of the task, and a CheckBox to mark the status of the task. Finally, two buttons "Save" and "Cancel" are arranged for the user to save or cancel the operation, creating a simple and user-friendly interface for task management.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".AddActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="What do you do?"
        android:textAlignment="center"
        style="@style/TextViewHeading"
    />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        style="@style/TextViewLabel"
        android:text="Task Content" />

    <EditText
        android:id="@+id/edtName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="text"
        android:layout_marginEnd="20sp"
        android:layout_marginStart="20sp"
        android:hint="Enter your content"
    />

    <TextView
        android:id="@+id/textView3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"

```

```

        style="@style/TextViewLabel"
        android:text="Due Date" />
<EditText
    android:id="@+id/edtDate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginEnd="20sp"
    android:layout_marginStart="20sp"
    android:ems="10"
    android:inputType="text"
    android:hint="Enter your due date" />
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal"
    android:weightSum="3">
    <TextView
        android:id="@+id/textView4"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        style="@style/TextViewLabel"
        android:layout_weight="1"
        android:text="Status" />
    <CheckBox
        android:id="@+id/cbStatus"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:layout_gravity="center_vertical"
        />
</LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_horizontal">
    <Button
        android:id="@+id/btnSaveAdd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        style="@style/CustomButtonStyle"
        android:layout_marginEnd="10dp"
        android:text="Save" />

    <Button
        android:id="@+id/btnCancelAdd"
        android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        style="@style/CustomButtonStyle"
        android:text="Cancel" />
    </LinearLayout>
</LinearLayout>

```

custom\_task\_list.xml: The XML code above defines an interface for a task list item in an Android application. The interface uses a vertical LinearLayout with a white background color and padding to create space between the elements. Inside, there is a main horizontal LinearLayout containing the task number, two TextViews for the task name and date, a TextView for the status, and a CheckBox to mark the item as deleted. This design makes it easy for users to track and manage their tasks.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="8dp"
    android:background="@android:color/white"
    >
    <!-- Main container -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="8dp"
        android:elevation="4dp"
        android:weightSum="6">
        <!-- Order Number TextView -->
        <TextView
            android:id="@+id/txtNumber"
            android:layout_width="0dp"
            android:layout_weight="1"
            android:layout_height="wrap_content"
            android:text="1."
            android:textSize="16sp"
            android:textColor="@android:color/black"
            android:layout_gravity="center_vertical"
            android:paddingEnd="12dp"
            android:fontFamily="sans-serif-medium" />
        <!-- Text container -->
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:paddingStart="12dp"
            android:layout_weight="3.5">
            <!-- Name TextView -->

```

```

<TextView
    android:id="@+id/txtTen"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Content"
    android:textSize="16sp"
    android:textColor="@android:color/black"
    android:fontFamily="sans-serif-medium" />
<!-- Date TextView -->
<TextView
    android:id="@+id/txtDate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Date"
    android:textSize="14sp"
    android:textColor="@android:color/darker_gray"
    android:fontFamily="sans-serif-light"
    android:paddingTop="4dp" />
</LinearLayout>
<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:paddingStart="12dp"
    android:layout_weight="1.5">
<!-- Name TextView -->
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Status"
    android:textSize="16sp"
    android:textColor="@android:color/black"
    android:fontFamily="sans-serif-medium" />
<!-- Date TextView -->
<TextView
    android:id="@+id/txtStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="False"
    android:textSize="14sp"
    android:textColor="@android:color/darker_gray"
    android:fontFamily="sans-serif-light"
    android:paddingTop="4dp" />
</LinearLayout>
<!-- Delete CheckBox -->
<CheckBox
    android:id="@+id/ck"
    android:layout_width="wrap_content"

```

```

        android:layout_weight="1"
        android:layout_height="wrap_content"
        android:layout_gravity="center_vertical"
    />
</LinearLayout>
</LinearLayout>

```

colors.xml: The colors.xml file defines the colors for the Android application.

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>
    <color name="red">#D50000</color>
    <color name="colorPrimary">#0091EA</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="colorLable">#373D3A</color>
</resources>

```

strings.xml: This strings.xml file contains string resources for Android applications.

```

<resources>
    <string name="app_name">Task Management App</string>
</resources>

```

themes.xml: This XML code defines the interface styles for the task management app, including properties for elements like TextView and Button. The Base.Theme.TaskManagementApp style uses the Material3 theme with the ability to switch between light and dark modes. The TextViewHeading and TextViewLabel styles define how the title and label are displayed with different colors, sizes, and fonts, helping to create clarity and readability. The CustomButtonStyle style gives buttons a primary background color and white text color, providing a better user experience. By defining these, the app can maintain consistency and aesthetics in its interface design.

```

<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.TaskManagementApp"
parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.TaskManagementApp"
parent="Base.Theme.TaskManagementApp" />
    <style name="TextViewHeading">
        <item name="android:textColor">@color/colorPrimary</item> <!-- Change to your
desired color -->
        <item name="android:textSize">22sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:padding">16dp</item> <!-- Add padding for better spacing -->
    </style>

```

```
        <item name="android:background">@drawable/textview_background</item> <!--  
Optional: custom background -->  
    </style>
```

```
    <style name="TextViewLabel">  
        <item name="android:textColor">@color/colorLable</item> <!-- Change to your desired  
color -->  
        <item name="android:textSize">20sp</item>  
        <item name="android:textStyle">bold</item>  
        <item name="android:paddingHorizontal">16dp</item> <!-- Add padding for better  
spacing -->  
        <item name="android:paddingTop">10dp</item>  
        <item name="android:paddingBottom">5dp</item>
```

```
    </style>
```

```
    <style name="CustomButtonStyle">  
        <item name="android:backgroundTint">@color/colorPrimary</item>  
        <item name="android:textColor">@android:color/white</item> <!-- White text for  
contrast -->  
        <item name="android:textSize">18sp</item>  
        <item name="android:textStyle">bold</item>  
        <item name="android:padding">10dp</item>  
    </style>  
</resources>
```